

Interface

A Java interface solves your multiple inheritance problem by giving you much of the polymorphic *benefits* of multiple inheritance without the pain and suffering from the Deadly Diamond of Death (DDD).

The way in which interfaces side-step the DDD is surprisingly simple: *make all the methods abstract!* That way, the subclass *must* implement the methods (remember, abstract methods *must* be implemented by the first concrete subclass), so at runtime the JVM isn't confused about *which* of the two inherited versions it's supposed to call.

Pet
<pre>abstract void beFriendly(); abstract void play();</pre>

A Java interface is like a 100% pure abstract class.

All methods in an interface are abstract, so any class that IS-A Pet **MUST** implement (i.e. override) the methods of Pet.

To DEFINE an interface:

```
public interface Pet {...}
```

Use the keyword "interface" instead of "class"

To IMPLEMENT an interface:

```
public class Dog extends Canine implements Pet {...}
```

Use the keyword "implements" followed by the interface name. Note that when you implement an interface you still get to extend a class

Making and Implementing the Pet interface

You say 'interface' instead of 'class' here

```
public interface Pet {  
    public abstract void beFriendly();  
    public abstract void play();  
}
```

interface methods are implicitly public and abstract, so typing in 'public' and 'abstract' is optional (in fact, it's not considered 'good style' to type the words in, but we did here just to reinforce it, and because we've never been slaves to fashion...)

All interface methods are abstract, so they **MUST** end in semicolons. Remember, they have no body!

Dog IS-A Animal
and Dog IS-A Pet

```
public class Dog extends Canine implements Pet {  
    public void beFriendly() {...}  
    public void play() {...}  
  
    public void roam() {...}  
    public void eat() {...}  
}
```

You say 'implements' followed by the name of the interface.

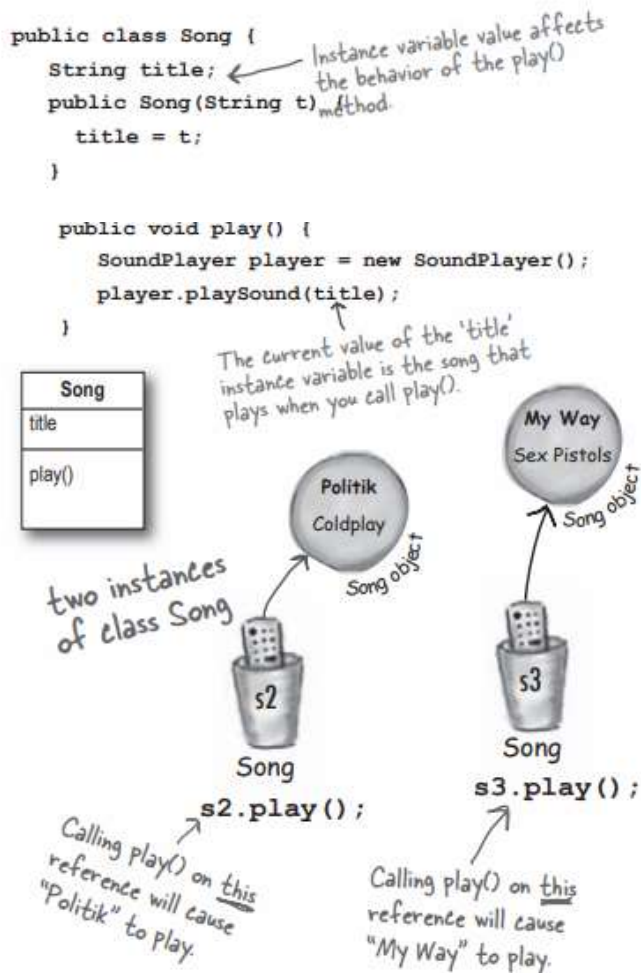
You SAID you are a Pet, so you **MUST** implement the Pet methods. It's your contract. Notice the curly braces instead of semicolons.

These are just normal overriding methods.

The difference between regular (non-static) and static methods

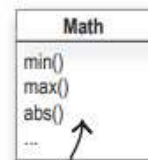
Java is object-oriented, but once in a while you have a special case, typically a utility method (like the Math methods), where there is no need to have an instance of the class. The keyword **static** lets a method run *without any instance of the class*. A static method means "behavior not dependent on an instance variable, so no instance/object is required. Just the class."

regular (non-static) method



static method

```
public static int min(int a, int b) {
    //returns the lesser of a and b
}
```



No instance variables. The method behavior doesn't change with instance variable state.

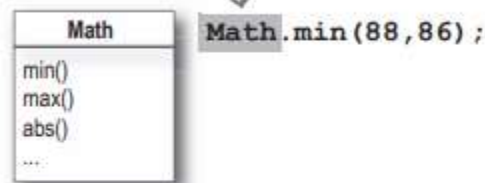
Math.min(42, 36);

Use the Class name, rather than a reference variable name.

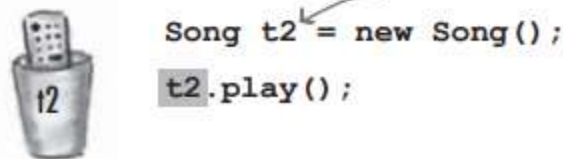


NO OBJECTS!!
Absolutely NO OBJECTS anywhere in this picture!

Call a static method using a class name



Call a non-static method using a reference variable name



What it means to have a class with static methods.

Often (although not always), a class with static methods is not meant to be instantiated. In Chapter 8 we talked about abstract classes, and how marking a class with the **abstract** modifier makes it impossible for anyone to say 'new' on that class type. In other words, *it's impossible to instantiate an abstract class*.

But you can restrict other code from instantiating a *non-abstract* class by marking the constructor **private**. Remember, a *method* marked private means that only code from within the class can invoke the method. A *constructor* marked private means essentially the same thing—only code from within the class can invoke the constructor. Nobody can say 'new' from *outside* the class. That's how it works with the `Math` class, for example. The constructor is private, you cannot make a new instance of `Math`. The compiler knows that your code doesn't have access to that private constructor.

This does *not* mean that a class with one or more static methods should never be instantiated. In fact, every class you put a `main()` method in is a class with a static method in it!

Typically, you make a `main()` method so that you can launch or test another class, nearly always by instantiating a class in `main`, and then invoking a method on that new instance.

So you're free to combine static and non-static methods in a class, although even a single non-static method means there must be *some* way to make an instance of the class. The only ways to get a new object are through 'new' or deserialization (or something called the Java Reflection API that we don't go into). No other way. But exactly *who* says new can be an interesting question, and one we'll look at a little later in this chapter.

Static variable: value is the same for ALL instances of the class

Imagine you wanted to count how many Duck instances are being created while your program is running. How would you do it? Maybe an instance variable that you increment in the constructor?

```
class Duck {  
    int duckCount = 0;  
    public Duck() {  
        duckCount++;  
    }  
}
```

this would always set duckCount to 1 each time a Duck was made

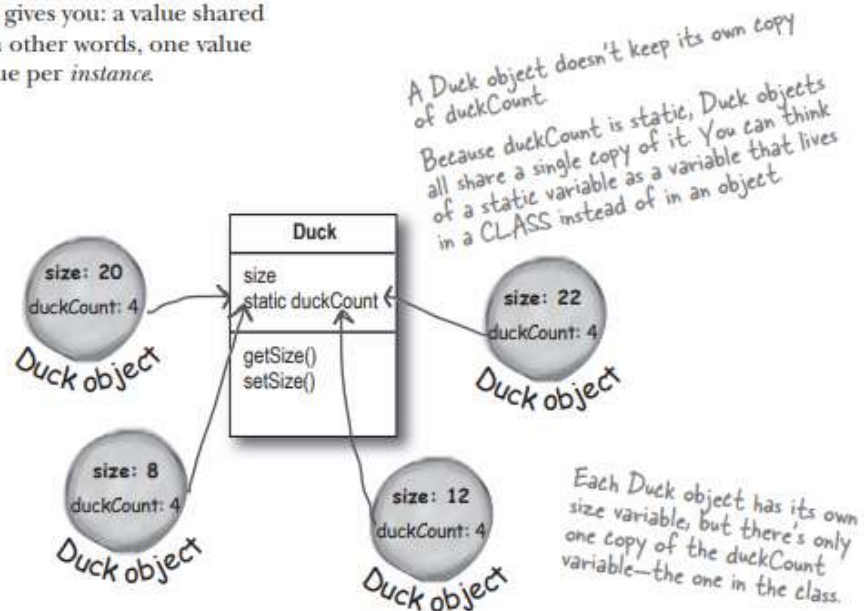
No, that wouldn't work because duckCount is an instance variable, and starts at 0 for each Duck. You could try calling a method in some other class, but that's kludgy. You need a class that's got only a single copy of the variable, and all instances share that one copy.

That's what a static variable gives you: a value shared by all instances of a class. In other words, one value per *class*, instead of one value per *instance*.

```
public class Duck {  
    private int size;  
    private static int duckCount = 0;  
  
    public Duck() {  
        duckCount++;  
    }  
  
    public void setSize(int s) {  
        size = s;  
    }  
    public int getSize() {  
        return size;  
    }  
}
```

The static duckCount variable is initialized ONLY when the class is first loaded, NOT each time a new instance is made.

Now it will keep incrementing each time the Duck constructor runs, because duckCount is static and won't be reset to 0.





Static variables are shared.

All instances of the same class share a single copy of the static variables.

instance variables: 1 per **instance**

static variables: 1 per **class**

Initializing a static variable

Static variables are initialized when a *class is loaded*. A class is loaded because the JVM decides it's time to load it. Typically, the JVM loads a class because somebody's trying to make a new instance of the class, for the first time, or use a static method or variable of the class. As a programmer, you also have the option of telling the JVM to load a class, but you're not likely to need to do that. In nearly all cases, you're better off letting the JVM decide when to *load* the class.

And there are two guarantees about static initialization:

Static variables in a class are initialized before any *object* of that class can be created.

Static variables in a class are initialized before any *static method* of the class runs.

```
class Player {
    static int playerCount = 0;
    private String name;
    public Player(String n) {
        name = n;
        playerCount++;
    }
}

public class PlayerTestDrive {
    public static void main(String[] args) {
        System.out.println(Player.playerCount);
        Player one = new Player("Tiger Woods");
        System.out.println(Player.playerCount);
    }
}
```

The playerCount is initialized when the class is loaded. We explicitly initialized it to 0, but we don't need to since 0 is the default value for ints. Static variables get default values just like instance variables.

Default values for declared but uninitialized static and instance variables are the same:

primitive integers (long, short, etc.): 0

primitive floating points (float, double): 0.0

boolean: false

object references: null

Access a static variable just like a static method—with the class name.

Static variables are initialized when the class is loaded. If you don't explicitly initialize a static variable (by assigning it a value at the time you declare it), it gets a default value, so int variables are initialized to zero, which means we didn't need to explicitly say "playerCount = 0". Declaring, but not initializing, a static variable means the static variable will get the default value for that variable type, in exactly the same way that instance variables are given default values when declared.

All static variables in a class are initialized before any object of that class can be created.

```
File Edit Window Help What?
% java PlayerTestDrive
0 ← before any instances are made
1 ← after an object is created
```

static final variables are constants

A variable marked **final** means that—once initialized—it can never change. In other words, the value of the static final variable will stay the same as long as the class is loaded. Look up `Math.PI` in the API, and you'll find:

```
public static final double PI = 3.141592653589793;
```

The variable is marked **public** so that any code can access it.

The variable is marked **static** so that you don't need an instance of class `Math` (which, remember, you're not allowed to create).

The variable is marked **final** because `PI` doesn't change (as far as Java is concerned).

There is no other way to designate a variable as a constant, but there is a naming convention that helps you to recognize one.

Constant variable names should be in all caps!

A static initializer is a block of code that runs when a class is loaded, before any other code can use the class, so it's a great place to initialize a static final variable.

```
class Foo {  
    final static int x;  
    static {  
        x = 42;  
    }  
}
```

Initialize a **final** static variable:

- ① At the time you declare it:

```
public class Foo {  
    public static final int FOO_X = 25;  
}
```

notice the naming convention -- static final variables are constants, so the name should be all uppercase, with an underscore separating the words

OR

- ② In a static initializer:

```
public class Bar {  
    public static final double BAR_SIGN;  
    static {  
        BAR_SIGN = (double) Math.random();  
    }  
}
```

this code runs as soon as the class is loaded, before any static method is called and even before any static variable can be used.

If you don't give a value to a final variable in one of those two places:

```
public class Bar {  
    public static final double BAR_SIGN;  
}
```

no initialization!

The compiler will catch it:

```
File Edit Window Help Jack-in  
% javac Bar.java  
Bar.java:1: variable BAR_SIGN  
might not have been initialized  
1 error
```


final isn't just for static variables...

You can use the keyword **final** to modify non-static variables too, including instance variables, local variables, and even method parameters. In each case, it means the same thing: the value can't be changed. But you can also use final to stop someone from overriding a method or making a subclass.

non-static final variables

```
class Poof {  
    final int size = 3; ← now you can't change size  
    final int whuffie;  
  
    Poof() {  
        whuffie = 42; ← now you can't change whuffie  
    }  
  
    void doStuff(final int x) {  
        // you can't change x  
    }  
  
    void doMore() {  
        final int z = 7;  
        // you can't change z  
    }  
}
```

final method

```
class Poof {  
    final void calcWhuffie() {  
        // important things  
        // that must never be overridden  
    }  
}
```

final class

```
final class MyMostPerfectClass {  
    // cannot be extended  
}
```

A **final variable** means you can't change its value.

A **final method** means you can't override the method.

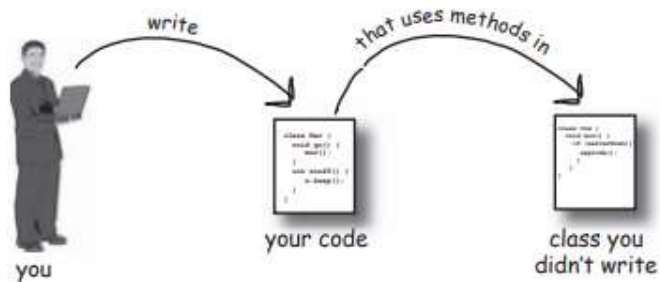
A **final class** means you can't extend the class (i.e. you can't make a subclass).



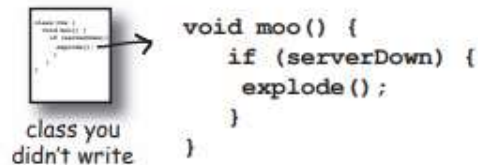
Exception Handling

What happens when a method you want to call (probably in a class you didn't write) is risky?

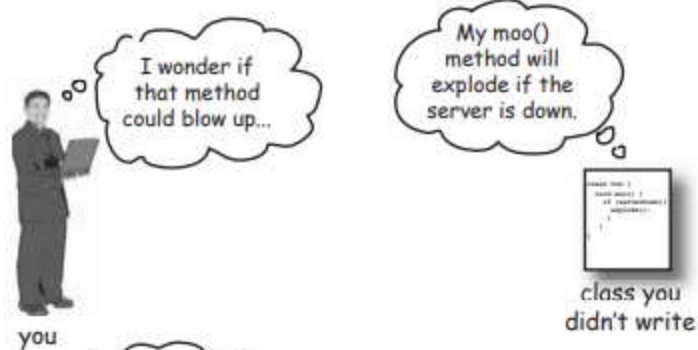
- ① Let's say you want to call a method in a class that you didn't write.



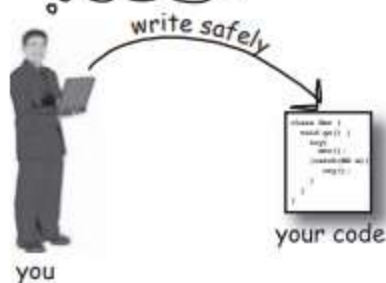
- ② That method does something risky, something that might not work at runtime.



- ③ You need to know that the method you're calling is risky.



- ④ You then write code that can handle the failure if it *does* happen. You need to be prepared, just in case.



Methods in Java use *exceptions* to tell the calling code, “Something Bad Happened. I failed.”

Java’s exception-handling mechanism is a clean, well-lighted way to handle “exceptional situations” that pop up at runtime; it lets you put all your error-handling code in one easy-to-read place. It’s based on you *knowing* that the method you’re calling is risky (i.e. that the method *might* generate an exception), so that you can write code to deal with that possibility. If you *know* you might get an exception when you call a particular method, you can be *prepared* for—possibly even *recover* from—the problem that caused the exception.

So, how *do* you know if a method throws an exception? You find a **throws** clause in the risky method’s declaration.

The compiler needs to know that **YOU** know you’re calling a risky method.

If you wrap the risky code in something called a **try/catch**, the compiler will relax.

A try/catch block tells the compiler that you *know* an exceptional thing could happen in the method you’re calling, and that you’re prepared to handle it. That compiler doesn’t care *how* you handle it; it cares only that you say you’re taking care of it.

An exception is an object... of type `Exception`.

Which is fortunate, because it would be much harder to remember if exceptions were of type `Broccoli`.

Remember from your polymorphism chapters that an object of type `Exception` *can* be an instance of any *subclass* of `Exception`.

Because an `Exception` is an object, what you *catch* is an object. In the following code, the **catch** argument is declared as type `Exception`, and the parameter reference variable is `ex`.

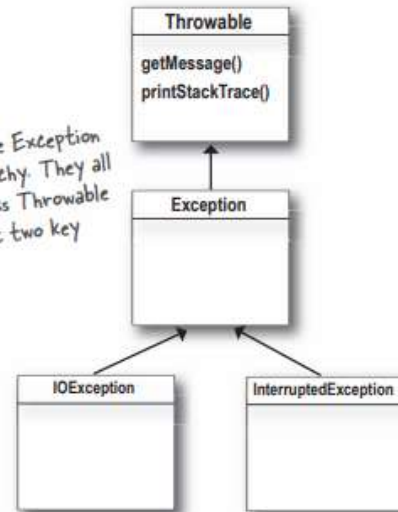
```
try {  
    // do risky thing  
} catch(Exception ex) {  
    // try to recover  
}
```

it's just like declaring a method argument.

This code only runs if an `Exception` is thrown.

What you write in a catch block depends on the exception that was thrown. For example, if a server is down you might use the catch block to try another server. If the file isn't there, you might ask the user for help finding it.

Part of the `Exception` class hierarchy. They all extend class `Throwable` and inherit two key methods.

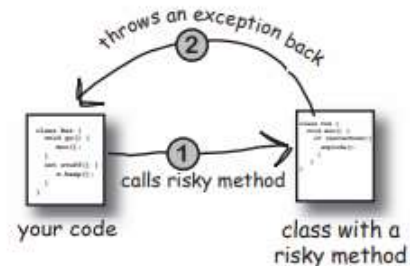


If it's your code that catches the exception, then whose code throws it?

You'll spend much more of your Java coding time *handling* exceptions than you'll spend *creating* and *throwing* them yourself. For now, just know that when your code *calls* a risky method—a method that declares an exception—it's the risky method that *throws* the exception back to *you*, the caller.

In reality, it might be you who wrote both classes. It really doesn't matter who writes the code... what matters is knowing which method *throws* the exception and which method *catches* it.

When somebody writes code that could throw an exception, they must *declare* the exception.



① Risky, exception-throwing code:

```
public void takeRisk() throws BadException {  
    if (abandonAllHope) {  
        throw new BadException();  
    }  
}
```

create a new Exception object and throw it

this method MUST tell the world (by declaring) that it throws a BadException

One method will catch what another method throws. An exception is always thrown back to the caller.

The method that throws has to declare that it might throw the exception.

② Your code that calls the risky method:

```
public void crossFingers() {  
    try {  
        anObject.takeRisk();  
    } catch (BadException ex) {  
        System.out.println("Aaargh!");  
        ex.printStackTrace();  
    }  
}
```

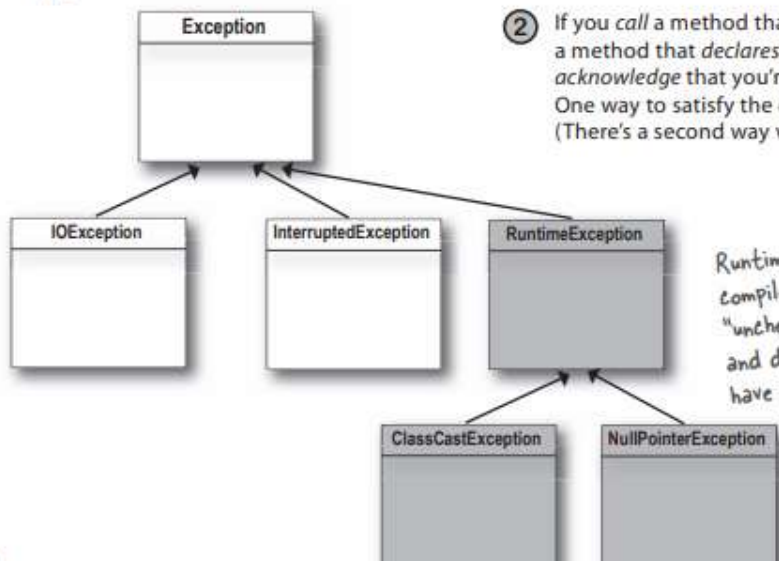
If you can't recover from the exception, at LEAST get a stack trace using the `printStackTrace()` method that all exceptions inherit

The compiler checks for everything except RuntimeExceptions.

The compiler guarantees:

- ① If you throw an exception in your code you *must* declare it using the `throws` keyword in your method declaration.
- ② If you *call* a method that throws an exception (in other words, a method that *declares* it throws an exception), you must *acknowledge* that you're aware of the exception possibility. One way to satisfy the compiler is to wrap the call in a try/catch. (There's a second way we'll look at a little later in this chapter.)

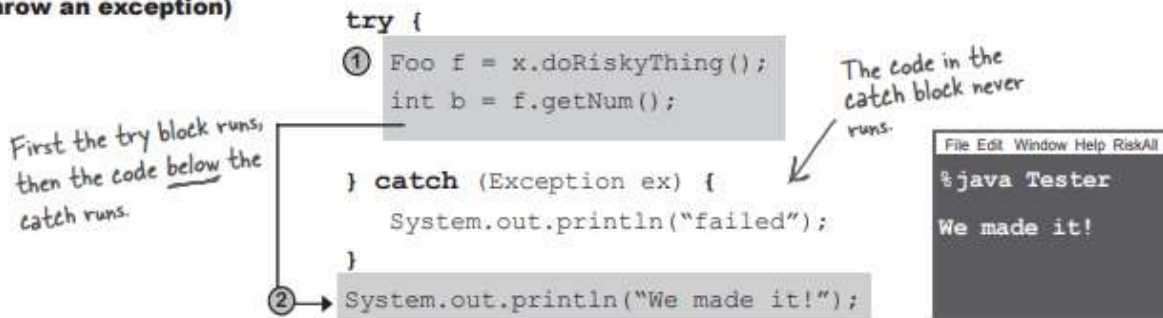
Exceptions that are NOT subclasses of RuntimeException are checked for by the compiler. They're called "checked exceptions"



RuntimeExceptions are NOT checked by the compiler. They're known as (big surprise here) "unchecked exceptions". You can throw, catch, and declare RuntimeExceptions, but you don't have to, and the compiler won't check.

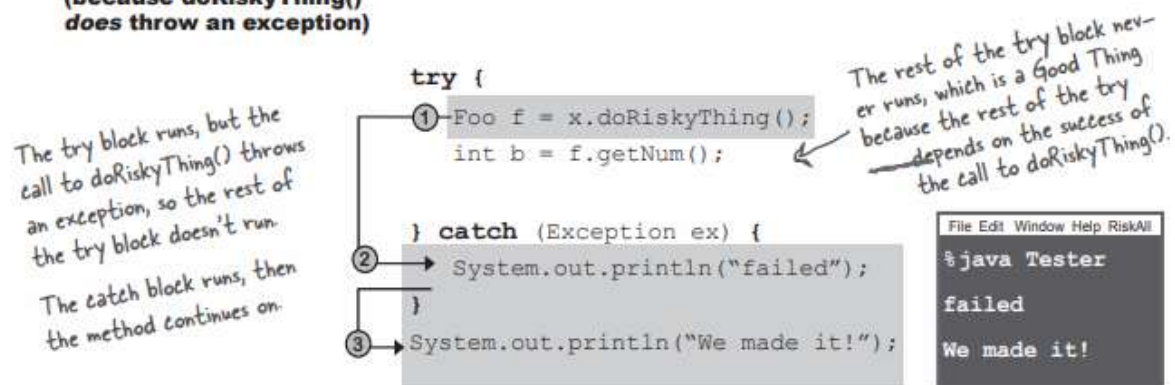
If the try succeeds

(doRiskyThing() does *not* throw an exception)



If the try fails

(because doRiskyThing() *does* throw an exception)



Finally: for the things you want to do *no matter what*.

If you try to cook something, you start by turning on the oven.

If the thing you try is a complete **failure**, *you have to turn off the oven.*

If the thing you try **succeeds**, *you have to turn off the oven.*

You have to turn off the oven no matter what!

A finally block is where you put code that must run *regardless* of an exception.

```
try {
    turnOvenOn();
    x.bake();
} catch (BakingException ex) {
    ex.printStackTrace();
} finally {
    turnOvenOff();
}
```

Without finally, you have to put the `turnOvenOff()` in *both* the try and the catch because *you have to turn off the oven no matter what*. A finally block lets you put all your important cleanup code in *one* place instead of duplicating it like this:

```
try {
    turnOvenOn();
    x.bake();
    turnOvenOff();
} catch (BakingException ex) {
    ex.printStackTrace();
    turnOvenOff();
}
```



If the try block fails (an exception), flow control immediately moves to the catch block. When the catch block completes, the finally block runs. When the finally block completes, the rest of the method continues on.

If the try block succeeds (no exception), flow control skips over the catch block and moves to the finally block. When the finally block completes, the rest of the method continues on.

If the try or catch block has a return statement, finally will still run! Flow jumps to the finally, then back to the return.

Just because you CAN catch everything with one big super polymorphic catch, doesn't always mean you SHOULD.

You *could* write your exception-handling code so that you specify only *one* catch block, using the supertype `Exception` in the catch clause, so that you'll be able to catch *any* exception that might be thrown.

```
try {  
    laundry.doLaundry();  
} catch(Exception ex) {  
    // recovery code...  
}
```

Recovery from WHAT? This catch block will catch ANY and all exceptions, so you won't automatically know what went wrong.

Write a different catch block for each exception that you need to handle uniquely.

For example, if your code deals with (or recovers from) a `TeeShirtException` differently than it handles a `LingerieException`, write a catch block for each. But if you treat all other types of `ClothingException` in the same way, then add a `ClothingException` catch to handle the rest.

```
try {  
    laundry.doLaundry();  
  
} catch(TeeShirtException tex) {  
    // recovery from TeeShirtException  
  
} catch(LingerieException lex) {  
    // recovery from LingerieException  
  
} catch(ClothingException cex) {  
    // recovery from all others  
}
```

  TeeShirtExceptions and LingerieExceptions need different recovery code, so you should use different catch blocks.

  All other ClothingExceptions are caught here.

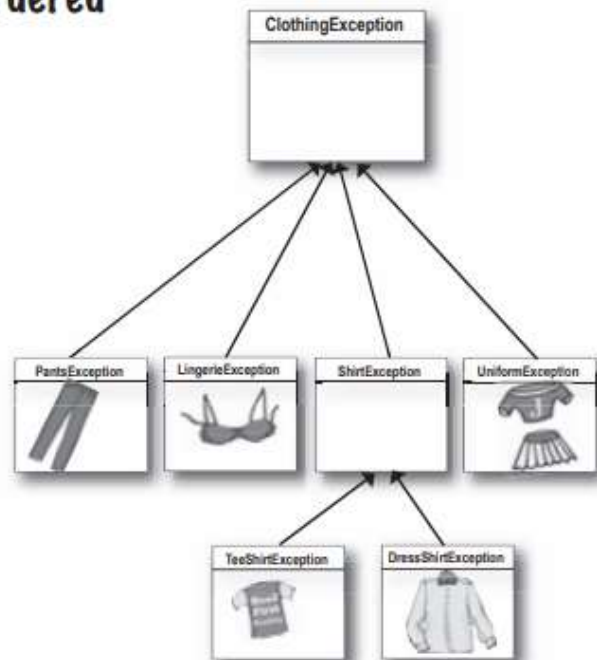
Multiple catch blocks must be ordered from smallest to biggest

 TeeShirtExceptions are caught here, but no other exceptions will fit.

```
catch(TeeShirtException tex)
```

 TeeShirtExceptions will never get here, but all other ShirtException subclasses are caught here.

```
catch(ShirtException sex)
```



When you don't want to handle an exception...

just duck it

If you don't want to handle an exception, you can duck it by declaring it.

When you call a risky method, the compiler needs you to acknowledge it. Most of the time, that means wrapping the risky call in a try/catch. But you have another alternative, simply *duck* it and let the method that called you catch the exception.

It's easy—all you have to do is *declare* that you throw the exceptions. Even though, technically, you aren't the one doing the throwing, it doesn't matter. You're still the one letting the exception whiz right on by.

But if you duck an exception, then you don't have a try/catch, so what happens when the risky method (`doLaundry()`) *does* throw the exception?



Act
Go t

When a method throws an exception, that method is popped off the stack immediately, and the exception is thrown to the next method down the stack—the *caller*. But if the *caller* is a *ducker*, then there's no catch for it so the *caller* pops off the stack immediately, and the exception is thrown to the next method and so on... where does it end? You'll see a little later.

```
public void foo() throws ReallyBadException {  
    // call risky method without a try/catch  
    laundry.doLaundry();  
}
```

← You don't REALLY throw it, but since you don't have a try/catch for the risky method you call, YOU are now the "risky method". Because now, whoever calls YOU has to deal with the exception.